HTML

Advanced HTML, XHTML and XML

Spolufinancováno Evropskou unií

MINISTERSTVO ŠKOLSTVÍ, MLÁDEŽE A TĚLOVÝCHOVY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

doc. Ing. Radek Burget, Ph.D.

burgetr@vut.cz

Ing. Jiří Hynek, Ph.D.

hynek@vut.cz

Outline

- Advanced HTML features
  - Multimedia content (audio, video)
  - Document metadata
- Alternatives to HTML
  - XML – General extensible markup language
  - XHTML – XML serialization of HTML markup

Multimedia `<audio>`

- The `controls` attribute displays the controls (play, stop, pause)
- JavaScript API for controlling the playback

```
<audio controls>
   <source src="sound.ogg" type="audio/ogg">
   <source src="sound.mp3" type="audio/mpeg">
   Audio is not supported.
</audio>
```

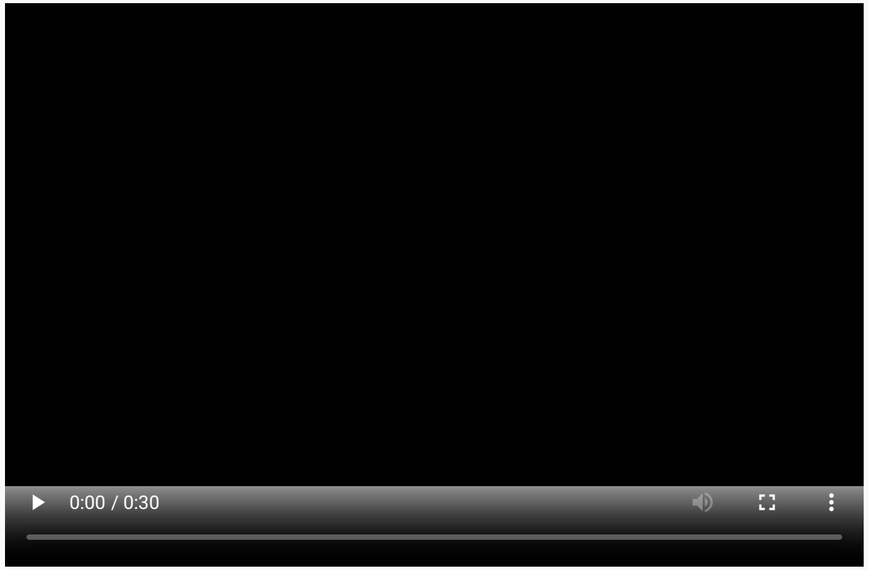
0:00 / 0:45

- Different supported formats and codecs (MP3, WAW, OGG, …)
- Browser-dependent

Multimedia <video>

```
<video controls height="240" width="320">
   <source src="file.mp4" type="video/mp4">
   <source src="file.ogg" type="video/ogg">
   Video is not supported.
 </video>
```

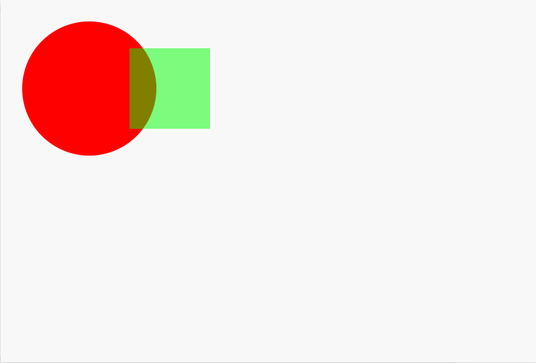
- Different supported formats and codecs (MP4, WebM, OGG, …)
- Browser-dependent



Vector Graphics <svg>

- SVG a MathML may be included directly in the document
  - with `xmlns` namespace definition optional in HTML
  - note the XML syntax discussed later

```
<svg height="120" xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="50" fill="red" />
  <rect x="80" y="20" height="60" width="60"
        style="stroke:none; fill:#00ff00;
        fill-opacity: 0.5;" />
</svg>
```



- Or use the SVG files as any other image

```
<img width="200" height="120" src="figure.svg">
```

Inline frame

New Document

Any document can be displayed in this frame.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi.

Etiam ac nunc vitae libero malesuada tempor. Curabitur laoreet, purus vitae vestibulum sollicitudin, dolor libero lobortis erat, id accumsan orci nibh vitae leo. Aenean suscipit velit vitae ex interdum, ac bibendum nulla facilisis. Vivamus gravida dolor lacinia, mollis elit nec, varius tortor. Ut ut nulla ac odio suscipit dictum. Cras at elit efficitur, viverra ipsum eu, condimentum nulla. Pellentesque sollicitudin, elit in ullamcorper semper, est erat tristique orci, in euismod nisi felis in sem. Cras id mi ut turpis tempus elementum a sit amet lorem. Phasellus mauris arcu, accumsan euismod purus non, aliquam tincidunt sapien. Aliquam vitae maximus nibh. Fusce pulvinar sodales massa vel convallis. Aliquam erat volutpat. Etiam vitae elementum sapien. Donec a feugiat enim.

Integer suscipit nunc ac lorem feugiat consectetur. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. In lobortis erat et tincidunt elementum. Morbi ut eleifend velit, nec condimentum risus. Fusce nec

```html
<iframe width="400" height="400" src="iframedoc.html">
Inline frames are not supported.
</iframe>
```

- Document is displayed in a separate context; it should not be able to access or influence the parent document.

ARIA

- **Accessible Rich Internet Applications** — W3C standard
- Adds accessibility semantics for assistive technologies (screen readers, etc.)
- Does not change visual appearance — purely for programmatic exposure
- `role` — declares the element's purpose

```
<div role="navigation">…</div>
<div role="dialog" aria-modal="true">…</div>
```

- `aria-*` **state and property attributes**

```
<button aria-expanded="false" aria-controls="menu">Menu</button>
<input aria-required="true" aria-describedby="hint">
<span id="hint">Must be a valid email</span>
```

ARIA: Landmark Roles

- Landmarks divide the page into navigable regions — screen readers let users jump between them

Role	Equivalent element	Purpose
`banner`	`<header>`	Site-wide header
`navigation`	`<nav>`	Navigation links
`main`	`<main>`	Primary content
`complementary`	`<aside>`	Supporting content
`contentinfo`	`<footer>`	Site-wide footer
`search`	`<search>`	Search widget
`form`	`<form>`	Form region

- Prefer **semantic HTML elements** — they carry the role implicitly; use `role` only when semantics can't be expressed otherwise

Structured Data & the Semantic Web

- Goal: embed **machine-readable semantics** into HTML pages
- Based on **RDF** (Resource Description Framework) — W3C standard
  - Data model: subject – predicate – object triples
  - Resources identified by URIs
- **Schema.org** — shared vocabulary of types and properties
  - Maintained by Google, Microsoft, Apple, Yandex
  - Covers: people, organizations, products, events, reviews, …
  - Used as the standard `itemtype` / `@type` vocabulary

Microdata

- Add semantics to HTML elements using specific attributes
- Attributes:
  - **itemscope** – item set container
  - **itemprop** – a property
  - **itemtype** – element type denoted as URL
  - **itemid** – a global identifier (e.g. ISBN for books)
  - **itemref** – global identifier reference
- Example using Schema.org vocabulary

```
<div itemscope="" itemtype="https://schema.org/Person">
 <p>My name is <span itemprop="givenName">John</span>.</p>
 <p>My surname is <span itemprop="familyName">Smith</span>.</p>
 <p>I was born in <span itemprop="birthPlace">Brno</span>.</p>
</div>
```

JSON-LD

- **JSON for Linking Data** — W3C recommendation, preferred by Google
- Embedded in a `<script>` tag — **no need to annotate HTML markup**
- Cleaner separation of data and presentation

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Person",
  "givenName": "John",
  "familyName": "Smith",
  "birthPlace": "Brno",
  "url": "https://example.org/john"
}
</script>
```

- `@context` — declares the vocabulary (Schema.org)
- `@type` — the type of the described entity
- Other keys are Schema.org property names

RDFa

- **RDF in Attributes** — W3C recommendation, extends HTML with RDF attributes
- More expressive than Microdata (full RDF support), but more complex syntax
- Key attributes: `vocab`, `typeof`, `property`, `resource`, `prefix`

```html
<div vocab="https://schema.org/" typeof="Person">
  <p>My name is
    <span property="givenName">John</span>
    <span property="familyName">Smith</span>.
  </p>
  <p>I was born in
    <span property="birthPlace">Brno</span>.
  </p>
</div>
```

Structured Data Consumers

- **Google Search** — rich results: star ratings, FAQs, breadcrumbs, events, products, recipes
  - Supports JSON-LD (preferred), Microdata, RDFa
  - Google Rich Results Test
- **Bing** — similar rich results in search
- **Social media** — related concept using `<meta>` tags:
  - Open Graph (Facebook, LinkedIn) — `og:title`, `og:image`, …
  - Twitter Cards — `twitter:card`, `twitter:title`, …
- **Knowledge graphs** — Wikidata, DBpedia use linked data principles

Document header: Metadata and relations

Link relations

- Adds semantics to links – the purpose of the link
- Applicable to the `<link>` and `<a>` elements
- A link to an external resource – adding related contents to a document

```
<link rel="stylesheet"  href="style.css" type="text/css">
```

- Hyperlink – navigation to another document

```
<a rel="next" href="slide010.html">next</a>
```

Link relations (II)

- Some interesting rel values:
  - **author**
  - **alternate** – alternative version (RSS, mobile, etc.)
  - **external** – external resources
  - **licence** – content license
  - **tag** – tag page (e.g. blogs)
  - **search** – search page
  - **first, last** – first, last document in a set
  - **prev, next** – previous, next document in a set
  - **up, down** – hierarchical navigation

Page icon

- Displayed in the URL field, in bookmarks, …
- A file in ICO format stored on server
- Specification in document header:

```
<link rel="icon" href="favicon.ico" type="image/x-icon">
```

- Most browsers automaticaly look for a **favicon.ico** file in the server root directory even without this specification
- Other image formats may be supported (SVG, PNG, …)
- Multiple icons may be provided, the browsers chooses the last usable one.

RSS and ATOM export

- XML languages for content syndication
- Allows to subscribe to new content via a client software
  - A standalone application or a browser extension

```
 <link rel="alternate"
      type="application/atom+xml"
      title="Feed of recent questions"
      href="/feeds">

<link rel="alternate"
      type="application/rss+xml"
      title="Top News"
      href="http://my-web.com/rss/index.php">
```

Meta information

- Meta information – additional information about the document itself
- Always in the document header
- Two possible variants:
  - User information

```
<meta name="…" content="…">
```

  - HTTP header equivalent

```
<meta http-equiv="…" content="…">
```

User information

- Document description

```
<meta name="description" content="Short description">
```

- Keywords

```
<meta name="keywords" content="WWW, HTML, pages, design">
```

- Document author

```
<meta name="author" content="John Smith, john@smith.org">
```

User information (II)

- Generator (software)

```
<meta name="generator" content="Mozilla Composer">
```

- Information for indexing robots

```
<meta name="robots" content="index,follow">
<meta name="robots" content="noindex,nofollow">
```

- Responsive design settings

```
<meta name="viewport" content="width=device-width">
<meta name="viewport" content="user-scalable=no, initial-scale=1">
```

Facebook meta tags

- Tags for the Facebook Open Graph Protocol
- Allows to specify metadata for page sharing

```
<meta property="og:title" content="Page Title">
<meta property="og:description" content="Page description">
<meta property="og:site_name" content="my-website.com">
<meta property="og:url" content="http://my-website.com/info/">
<meta property="og:image"
      content="http://my-website.com/thumbnail.jpg">
<meta property="og:type" content="article">
```

HTTP equivalents

- The same iformation as in HTTP headers
- No access to the server configuration needed
- Content type = MIME type + encoding

```
<meta http-equiv="content-type" content="text/html; charset=utf-8">
```

- Content language

```
<meta http-equiv="content-language" content="en">
```

Redirect

- After displaying a page, another document is automaticaly displayed after the specified time
- E.g. redirect to IMDB.com after 6 seconds

```
<meta http-equiv="refresh" content="6;URL=https://www.imdb.com/">
```

Cache control

- Controls whether to store or not to store the document in cache
- Influences just the browser cache

```
<meta http-equiv="cache-control" content="cache">

<meta http-equiv="cache-control" content="no-cache">
```

- Cache expiration

```
<meta http-equiv="expires" content="Wed 19 Nov 2025 16:10:00">
```

- After this date, the page should be removed from cache

Script Loading: `defer` and `async`

- By default `<script>` **blocks** HTML parsing until the script is downloaded and executed
- `defer` — download in parallel, execute **after** the document is parsed
- `async` — download in parallel, execute **immediately** when ready (order not guaranteed)

```
<script src="app.js" defer></script>
<script src="analytics.js" async></script>
```

- Use **`defer`** for scripts that need the DOM (most application scripts)
- Use **`async`** for independent scripts (analytics, ads)
- Both attributes have no effect on inline scripts

Resource Hints

- Instruct the browser to load or prepare resources early
- Declared via `<link>` in `<head>`
- **`preload`** — fetch a critical resource immediately, before it is discovered in the page

```
<link rel="preload" href="hero.jpg" as="image">
<link rel="preload" href="font.woff2" as="font" crossorigin>
```

- **`prefetch`** — fetch a resource likely needed for the next navigation (low priority)

```
<link rel="prefetch" href="next-page.js">
```

Resource Hints (II)

- **`preconnect`** — open the TCP/TLS connection to an origin early

```html
<link rel="preconnect" href="https://fonts.googleapis.com">
```

- **`lazy loading`** — defer loading of off-screen images and iframes

```html
<img src="photo.jpg" loading="lazy" alt="…">
<iframe src="map.html" loading="lazy"></iframe>
```

Alternatives to HTML: XML and XHTML

XML and XHTML

- **XML**
  - A general standard for data and document exchange
  - Syntactically similar to HTML, does not define the meaining of the tags (it depends on the particular application)
- **XHTML**
  - The application of XML for web pages
  - Replace HTML with XML
  - Tag names and their meaning equal to HTML, syntax is XML

Advantages of XML

- Simple implementation
  - The language is not so complex
  - Quite easy to process
- It is possible to store, index and search in documents
  - Native XML databases
- Many tools for XML processing
  - Document parsing (DOM, SAX parsers)
  - Transformations (XSLT)
  - …
- Many applications already use XML

History

- XHTML 1.0 (2000)
  - Basic transformation of existing HTML 4 (last version 1999) to XML
  - Backward compatibility with HTML
- XHTML 1.1 (2001)
  - Modularization (modules for different applications)
  - Preparation for further extensions
  - Strict XML (no backward compatibility with HTML but the same tags)
- XHTML 2.0 (last working draft 2006)
  - Modern design but not compatible with predecessors
  - Slow developent, did not solve actual problems (e.g. multimedia), not well received
- HTML 5
  - Back to HTML, solve the important issues
  - Includes both the HTML syntax and the XHTML syntax

HTML and XHTML

- Common features
  - The same meaning of most the tags and attributes
  - Tags and attributes are written the same way
- Differences
  - Strict syntax
  - Formatting tags and attributes removed, replaced by CSS
  - It has to respect the XML rules
- Currently, use of XHTML is not recommended, but it works

HTML vs XHTML Differences

- Additional XML header (optional for UTF-8 encoding)

```
<?xml version="1.0" encoding="utf-8"?>
```

- The root element `<html>` should have the `xmlns` and `xml:lang` attributes

```
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en">
```

Writing tags

- All the tag names must be entered in **lowercase**
- All non-empty tags must have an end tag
- The tags empty **from definition** (`<br>`) must have an end tag too `<br></br>` or they can be written as `<br/>`

```
<img src="…" alt="…" />
<input type="…" name="…" />
```

- Scripts that must be marked as CDATA:

```
<script type="text/javascript">
   <![CDATA[
     … JavaScript code …
     ]]>
</script>
```

XHTML and the Transfer Protocol

- XHTML should be processed by a XML parser in the browser
  - Which is different from the default HTML parser
- This must be indicated by the MIME type during the HTTP transfer
  - Type `text/html` for HTML documents
  - Type `application/xhtml+xml` or `application/xml` for XHTML documents
- Practical issues
  - A server side configuration necessary
  - Sometimes the `.xhtml` extension is properly recognized
- Examples

General XML

- The browsers allows general XML processing (which is not XHTML)
- No built-in style is defined
  - But it is possible to define the author style sheet
  - The `xml-stylesheet` directive
- Style definition
  - **CSS**
    - Style for the individual elements
  - **XSLT**
    - Internal transformation to an HTML document
    - May include JavaScript etc.
- Examples

To be continued…